

Guía de Aprendizaje: Creación de una Matriz Dinámica

Malak Sanchez

Workshop 2026-II

16 de agosto de 2026

1. El reto: ¿Qué es una matriz dinámica?

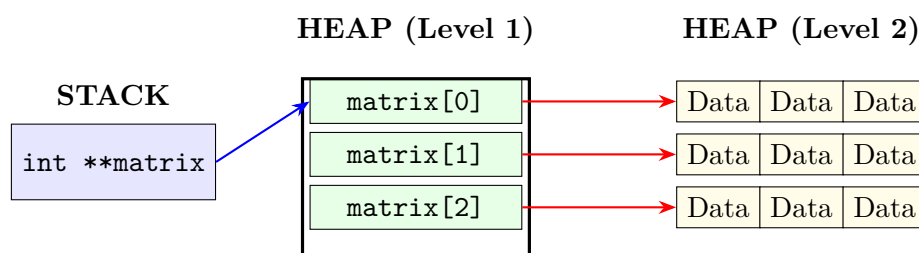
En C, una matriz en el Stack se declara como `int mat[F][C]`. Pero si queremos que el tamaño sea dinámico, no podemos hacerlo de forma tan directa. Necesitamos usar un **puntero a punteros** (`int **`).

2. Estructura Mental

Imagina una matriz dinámica no como un bloque sólido, sino como:

1. Un puntero principal (`int **matriz`).
2. Un arreglo de **punteros** (uno por cada fila).
3. Cada uno de esos punteros apunta a un arreglo de **enteros** (las columnas).

3. Dibujo Ilustrativo



4. Cómo se construye (El Código)

Debemos hacer la reserva en **capas**. Primero las filas, luego cada fila individual.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(){
5      int rows = 3, cols = 4;
6
7      // 1. Allocate the array of pointers (the rows)
8      int **matrix = (int **) malloc(rows * sizeof(int *));
9      if(!matrix){
10         printf("Error: Could not allocate memory.\n");
11         return 1;
12     }
13
14     // 2. For each row, allocate the array of data
15     for(int i=0; i < rows; i++){
16         matrix[i] = (int *) malloc(cols * sizeof(int));
17         if(!matrix[i]){
18             printf("Error: Could not allocate memory.\n");
19             return 1;
20         }
21     }
22
23     // 3. Normal usage: matrix[i][j]
24     matrix[1][2] = 50;
25
26     // 4. FREE (In inverse order of creation)
27     for(int i=0; i < rows; i++){
28         free(matrix[i]); // First we free each row
29     }
30     free(matrix); // At the end the "parent" of all
31
32     return 0;
33 }
```

5. Regla de Oro

Por cada `malloc` que hagas, debe existir un `free`. En una matriz de N filas, haces $N + 1$ allocations (`malloc`): una para el arreglo de punteros y N para las filas datos. Por lo tanto, ¡necesitas $N + 1$ llamadas a `free`!

6. Errores comunes

- **Liberar solo el padre:** Si haces `free(matrix)` sin liberar las filas primero, habrás perdido la dirección de las filas y esa memoria quedará "bloqueada" (Memory Leak).
- **Confusión en `sizeof`:** En el primer `malloc` se usa `sizeof(int *)`, porque estamos guardando punteros. En el segundo se usa `sizeof(int)`, porque guardamos enteros.